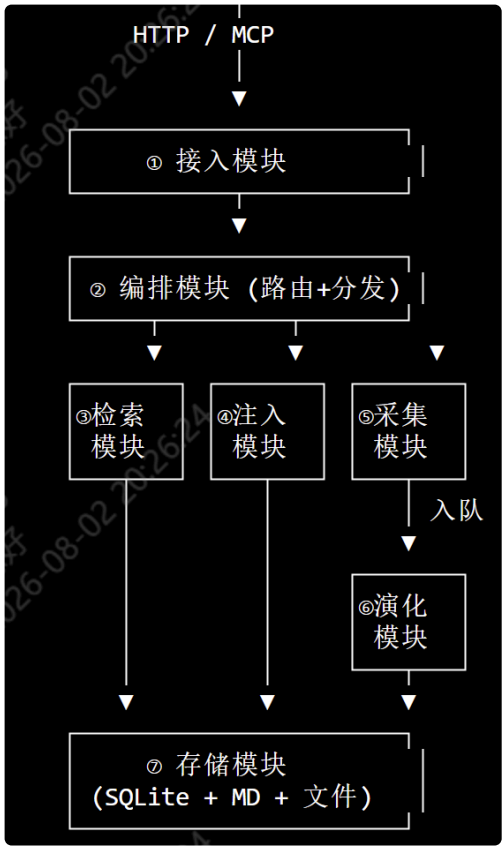


总体方案

- 架构
 - 数据分层
 - 数据类型
 - 接口工具
 - 对外接口
 - 内部函数 (POST /prompt)
 - 时序图
 - 实例接入
 - codegen
 - docgen
 - 框架通用性
 - 数据更新

架构

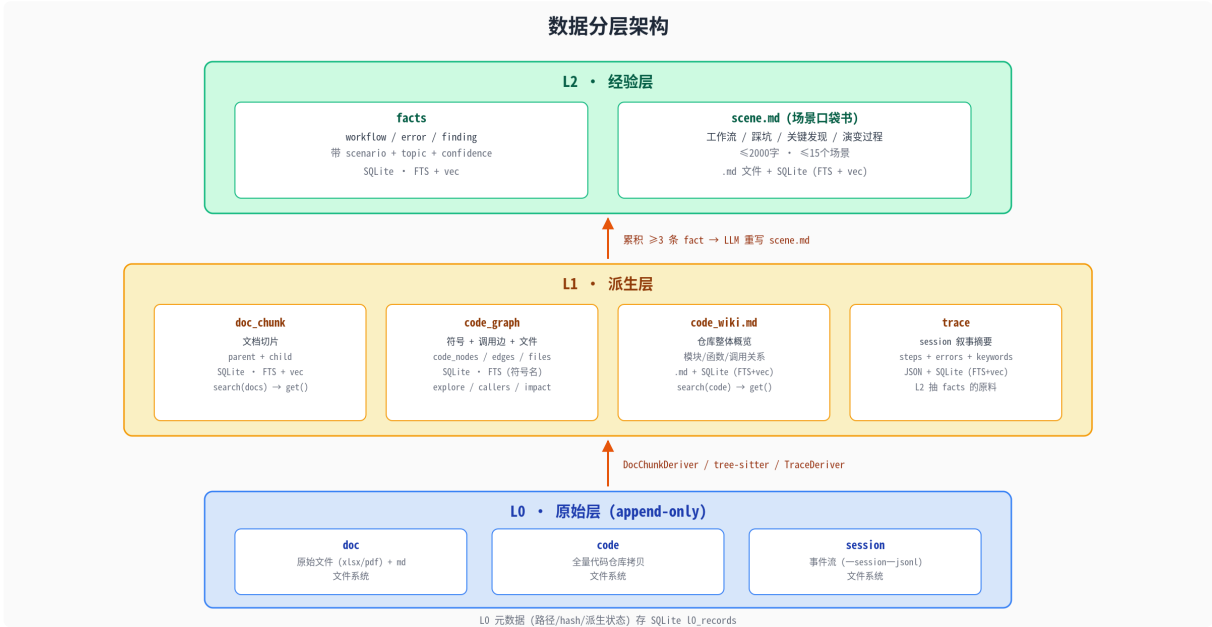


memory 架构分7个模块，外部 agent 请求 (http、MCP) 经**接入模块**到达**编排模块**（区分不同 workspace并转发）；查询问题走**检索模块**，system prompt 注入走**注入模块**，事件/文件写入走**采集模块**；数据入库后异步触发加工**演化模块**（L0-L1-Facts-scene）；所有读写操作都经过**存储模块**

模块	功能
----	----

接入模块	翻译外部的http、mcp协议请求（校验）
编排模块	workspace路由 + 请求分发
检索模块	search 混合检索 + get 全文获取（agent主动拉）
注入模块	常驻通用 scene + 任务相关 scene
采集模块	接收 agent运行事件 + 文件导入
演化模块	L0 — L1— Facts — scene
存储模块	SQL + 文件读写

数据分层



L0 原始数据：文档、代码、会话；file-system 存储，SQLite 存元数据

sql用处：查找；去重+变更检测；pipeline触发

需要做的事：设计sql字段，三种类型的元数据字段

L1 派生数据：文档chunk、codegraph、trace；SQLite 存 content+ embedding + FTS + vec

文档：v0只做chunk，后续再增加wiki，graph

code：codegraph（符号、边、文件、FTS表）+ wiki（md）

trace：LLM从几百条事件流中提炼出来的“这次任务干了什么”

L2 经验积累：facts、scene；SQLite 存 content+ embedding + FTS + vec

- facts：单条事实，分workflow、error、finding 三类，带场景标签
- scene：同场景下的 facts 聚合成该场景的口袋书（ workflow、踩坑、关键发现、演变过程）

facts 去重判断（skip、update、merge、store）；scene更新以 facts 新增的数量触发；通用场景定期扫描

数据类型

数据	存储位置	检索方式	查什么
facts	SQLite	FTS + vec	文本query --> 全文返回
doc chunk	SQLite	FTS + vec	文本query -> 摘要 -> get 全文
code wiki	.md文件 + SQLite	FTS + vec	文本query -> 摘要 -> get 全文
code 边	SQLite	SQL 直查	符号名 -> 关系查询
code 符号	SQLite	FTS (符号名)	符号名 -> 位置 + 源码
场景.md (数量限制)	.md文件 + SQLite	FTS + vec	文本query -> 摘要 -> get 全文

接口工具

对外接口

- 1.search(query, type) 按content长度判断给不给完整content
type: facts / docs / scenes / code
- 2.get 拿全文
- 3.code_explore 搜代码，返回完整源码
- 4.code_graph(symbol, relation, depth)
relation = callers 谁调了这个
relation = callees 这个调了谁
relation = impact 改这个影响什么
- 5.events 事件采集, plugin 调用

内部函数 (POST /prompt)

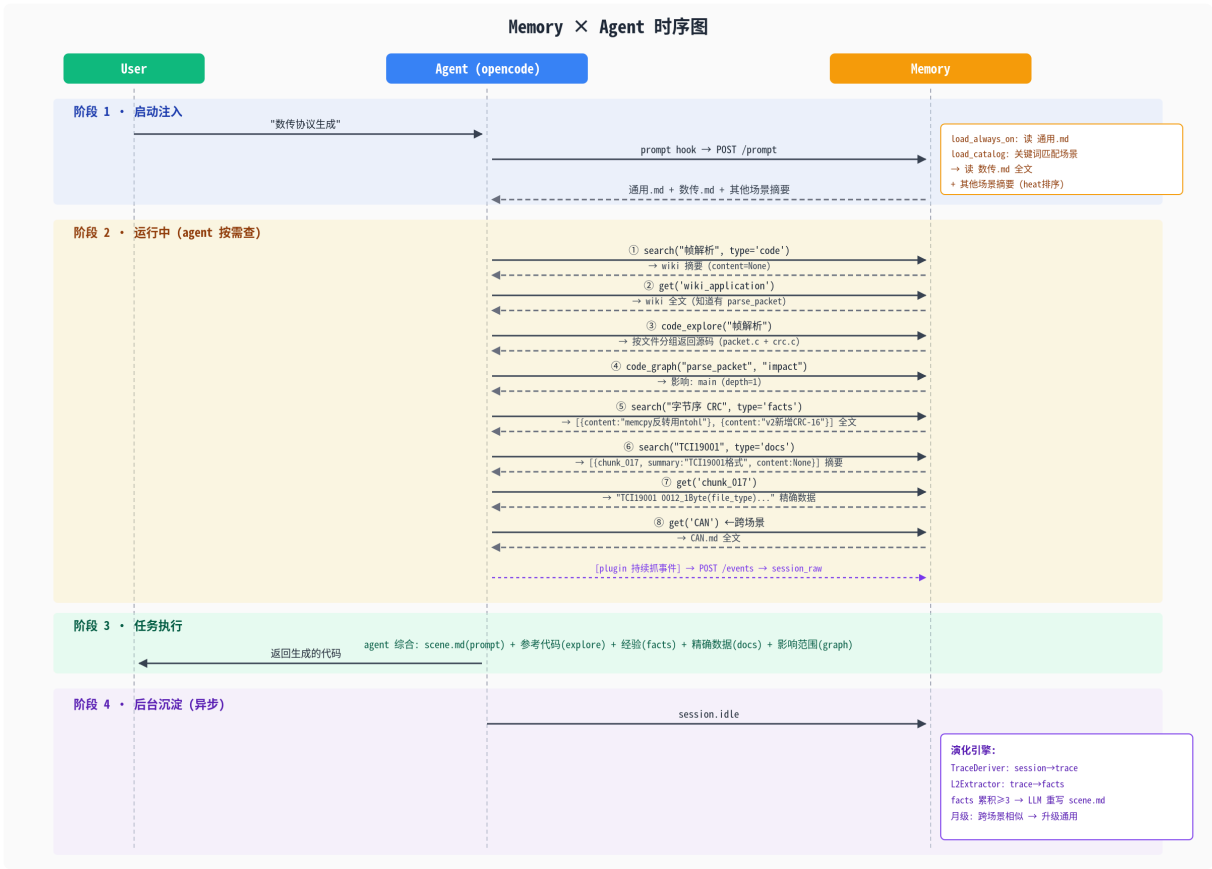
- 1.load_always_on (workspace)
直接读文件 “通用.md” , 返回全文
- 2.load_catalog (workspace, "query")
关键词匹配 scene.md
直接读文件 “FPGA.md” , 返回全文
其他场景：从 sql 找到摘要

时序图

启动时：注入服务

system prompt：通用.md 全文 + top-1 场景全文 + 其余场景摘要

运行时：agent 主动调用 search、get、code_explore、code_graph 这四个工具



实例接入

codegen

codegen在opencode上跑，opencode提供 plugin 机制和 MCP client

plugin: 1. event hook 抓事件 (http)

2.prompt hook 注入 system prompt (http)

MCP client 工具: search () 、 get () 、 code_explore () 、 code_graph ()

docgen

docgen 在 langgraph 上跑，langgraph 提供 callback

- 1. CallbackHandler 抓事件 (http)
- 2. memory_init 工具 拼入 system prompt (http)
- 3. langchain tools : search () 、 get ()

框架通用性

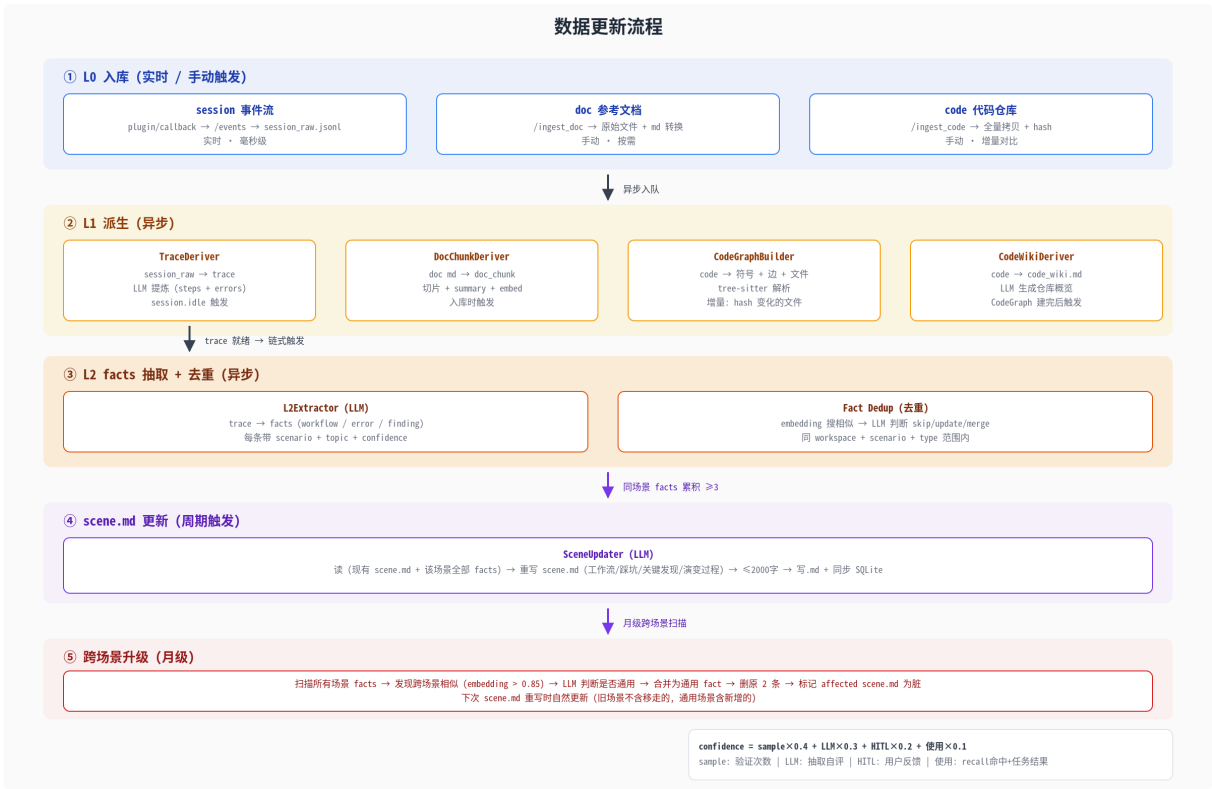
模块的位置、接口不变，但模块内部调可插拔组件

主要是演化模块和接收模块，可以自己实现+配yaml，框架本体不变

组件类型	作用	通用库
EventHandler	L0 事件流接收	OpencodeEventHandler / LanggraphEventHandler
L0Processor	L0导入	DocProcessor/CodeProcessor
L1Deriver	L0-L1派生	CodeAstDeriver / TraceDeriver / DocChunkDeriver
L2Extractor	L1 — Facts 抽取	TraceFactsExtractor
L2 Merger	facts → Scene	SceneUpdater

数据更新

抽取 —— 累积 —— 合并 —— 审批 —— 注入



1.facts 是 scene 的原料，但成熟度体现在 facts 的 confidence 里面

2.confidence 取决于 facts 数量，也取决于LLM自评分数

3.confidence 有反馈过程，下次被agent使用，根据结果反馈